



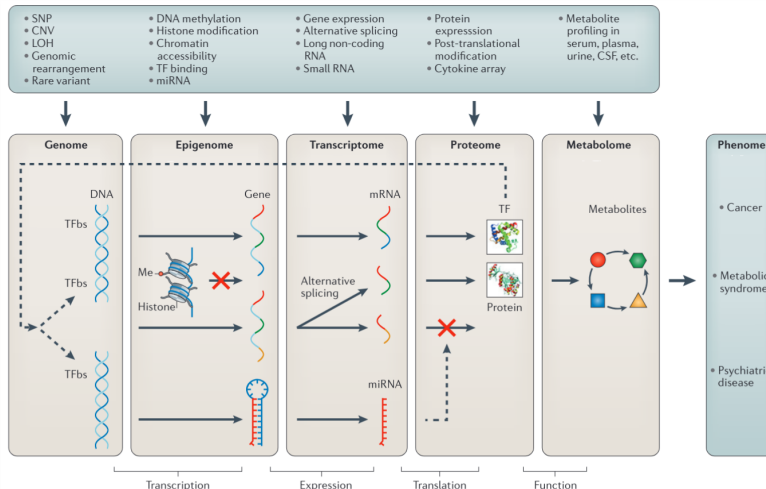
Accessible and Reproducible Analyses with the CoLoMoTo notebook

Pedro T. Monteiro

INESC-ID / IST - Universidade de Lisboa

WTAC 2025 - Computational Systems Biology

Cell is tightly **regulated** at several levels



(Ritchie et al., Nat. Rev. Gen. 2015)

Methods of integrating data to uncover genotype–phenotype interactions

[Marylyn D. Ritchie](#) , [Emily R. Holzinger](#), [Ruowang Li](#), [Sarah A. Pendergrass](#) & [Dokyoon Kim](#)

Nature Reviews Genetics **16**, 85–97 (2015) | [Cite this article](#)

102k Accesses | 592 Citations | 170 Altmetric | [Metrics](#)

Key Points

- Technological advances have vastly expanded the amount of omic data currently available. Historically, each type of data was analysed separately, although approaches to integrate omic data sets to predict complex phenotypic traits are now emerging.
- Such systems genomics approaches to combine multiple data types provide a more comprehensive understanding of complex genotype–phenotype associations than analysis of one data set.
- Data from multiple sources that point to the association of the same gene or pathway are less likely to result in false positives.
- There are various strengths and weaknesses of the available strategies. The approach used needs to be selected according to specific types of data, different types of scientific questions or different types of underlying genomic models.

Structural → Dynamical view

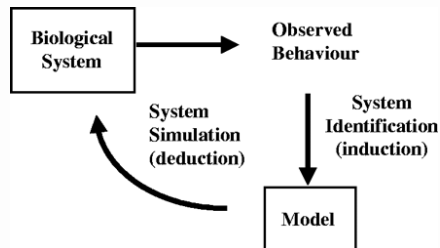
Network components need to be quantified:

- Values change over time (created / consumed)
⇒ We need something to describe the system' evolution!

(Kitano, *Science* 2002)

Rely on models to analyse networks

- Understand of the way in which particular molecular mechanisms control a cellular process
- Predict novel phenomena that can be confronted with experimental data



(King, Garrett and Coghill, *Bioinformatics* 2005)

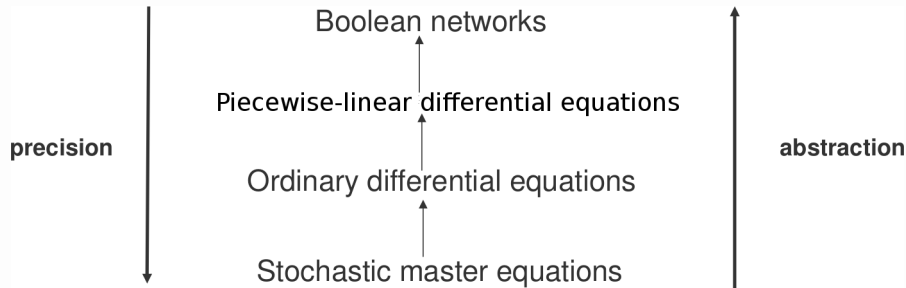
Formalisms

A variety of physical and mathematical approaches are used:

- Non-linear
- Piecewise-linear
- Logical (Boolean networks)
- Petri nets
- Statistical-mechanical
- Stochastic

Hierarchy of modeling formalisms

Describe the system at different levels of detail (e.g. continuous vs qualitative)



(de Jong, *J. Comput. Biol.* 2002) (Hasty et al., *Nat. Rev. Genet.* 2001)

(Smolen et al., *Bull. Mat. Biol.* 2000) (Szallasi et al., *Systems Modeling in Cellular Biology* 2006)

Well established qualitative formalisms for modeling gene regulatory networks:

- Piecewise-linear differential equations (PLDEs)
(Glass and Kauffman, *J. Theor. Biol.* 1973)
- **Logical formalism**
(Boolean networks)
(Thomas *et al.*, *Bull. Math. Biol.* 1995)
- Petri nets, ...
- Lack of quantitative data
(ON/OFF mechanisms, thresholds)
- Discrete time

Implicit assumptions

- Ignore intermediate gene products (mRNA)
- Ignore gene expression machinery (RNA polymerase, ribosome)
- Simplification of complex interactions of regulators with DNA to single response function

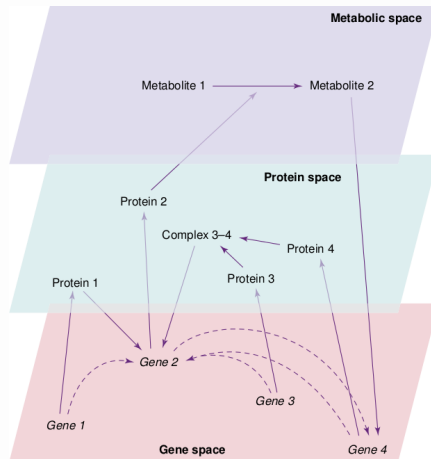
Gene regulatory networks

- consist of genes, gene products (RNA, proteins), and the regulatory effect of the latter on the expression of other genes

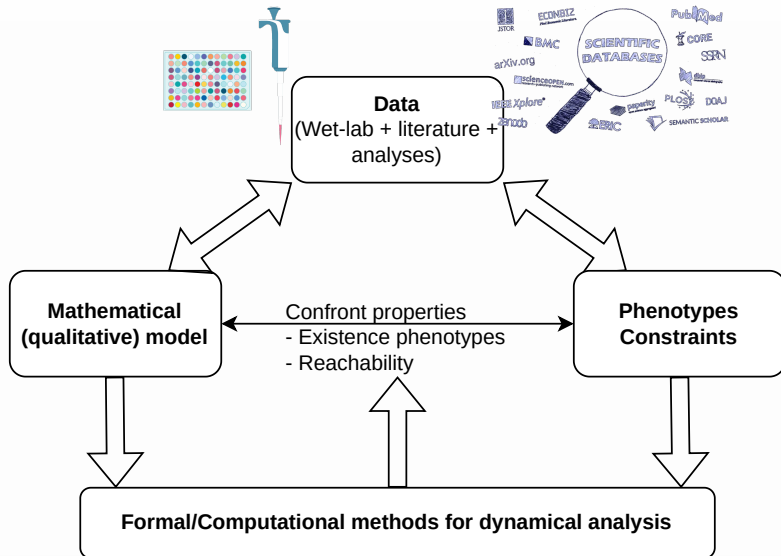
(Bolouri et al., *Computational Modeling of Gene Regulatory Networks* 2008)

- cannot be reduced to direct interactions (transcriptional regulation), but also include indirect interactions (mediated by metabolism)

(Brazhnik et al., *Trends Biotechnol* 2002)



Different levels



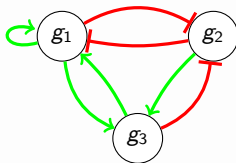
	Wet lab Lab book	Dry lab Program
Material	Biological material: - Cell type - Genetic background	Dataset: - Source - Format
Tools	Reagents & Equip.: - Provider, brand, ...	Software: - Version
Protocol	- Steps - Conditions	- Commands - Parameters

	Wet lab Lab book	Dry lab Program
Material	Biological material: - Cell type - Genetic background	Dataset: - Source - Format
Tools	Reagents & Equip.: - Provider, brand, ...	Software: - Version
Protocol	- Steps - Conditions	- Commands - Parameters

The (reproducibility) devil is in the details!

Regulatory graph

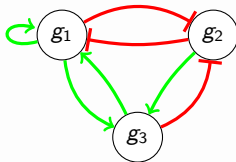
- Is a tuple (G, E) where G denotes a set of nodes/vertices (eg. gene expression) and E a set of edges
- An edge $(i, j) \in E$ indicates that i regulates the expression of j
- Edges can have sign information about interactions:
 - $(i, j, +)$ for “ i activates j ”
 - $(i, j, -)$ for “ i inhibits j ”



Boolean model (regulatory graph + logical rules) $\mathcal{M} = (G, K)$

(Thomas and d'Ari, *Biological Feedback* 1989)

- $G = \{g_i\}_{i=1,\dots,n}$ is a set of n regulatory components
- $\prod_{g_i \in G} \{0, 1\}$ defines the state space $\mathcal{S}$
- K is the set of regulatory functions $K_{g_i} \in K : \mathcal{S} \rightarrow \{0, 1\}$



$$K_{g_1}(x) = x_1 \vee (\neg x_1 \wedge \neg x_2) \vee (\neg x_1 \wedge x_3)$$

$$K_{g_2}(x) = \neg x_1 \vee \neg x_3$$

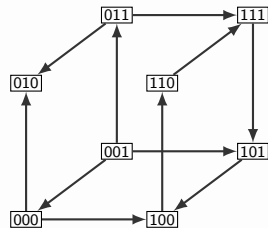
$$K_{g_3}(x) = x_1 \wedge x_2$$

- Gene expression has two (or more) levels: **0** (Inactive) and **1** (Active)
- K_{g_i} is a combination of basic Boolean operators: $\neg$, $\wedge$ and $\vee$

State transition graph (STG)

- Represents the dynamics of the Boolean model
- Nodes are states in $\mathcal{S}$
- Arcs $(i, j) \in \mathcal{S}^2$ denote transitions between states

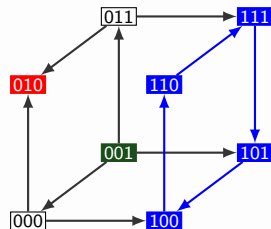
Successor state: $\forall_{g_i \in G}, g_i^{t+1} = K_{g_i}(g_1^t, \dots, g_n^t)$



State transition graph (STG)

- Represents the dynamics of the Boolean model
- Nodes are states in $\mathcal{S}$
- Arcs $(i, j) \in \mathcal{S}^2$ denote transitions between states

Successor state: $\forall_{g_i \in G}, g_i^{t+1} = K_{g_i}(g_1^t, \dots, g_n^t)$



Attractors

Correspond to (biologically relevant) asymptotic behaviours

- **Stable state:** all gene levels are maintained
e.g., differentiated states, death, ...
- **Complex attractor:** long-lasting oscillatory behaviour
e.g., cell cycle, ...

Interesting properties

- What are the attractors of the system? (stable states, complex attractors)
Is there multistability?
- Are these attractors reachable from initial conditions?
- Are these attractors maintained under input variations?
- What is the likelihood of reaching an attractor from a given portion of the state space?
- ...

Combinatorial explosion of number of states

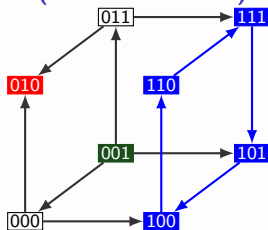
# Components	# States	
	Boolean	3-valued
3	8	27
10	1 024	59 049
20	1 048 576	3 486 784 401
30	1 073 741 824	205 891 132 094 649
40	1 099 511 627 776	12 157 665 459 056 928 801

Combinatorial explosion of number of states

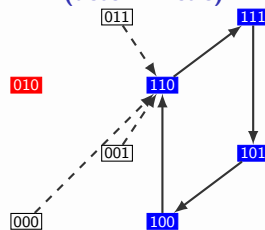
# Components	# States	
	Boolean	3-valued
3	8	27
10	1 024	59 049
20	1 048 576	3 486 784 401
30	1 073 741 824	205 891 132 094 649
40	1 099 511 627 776	12 157 665 459 056 928 801

- As the model size increases $\Rightarrow$ manually intractable
- State transitions graphs are not practical for most current models
- **Logical models enable efficient analytical methods**

Asynchronous updating (non-deterministic)

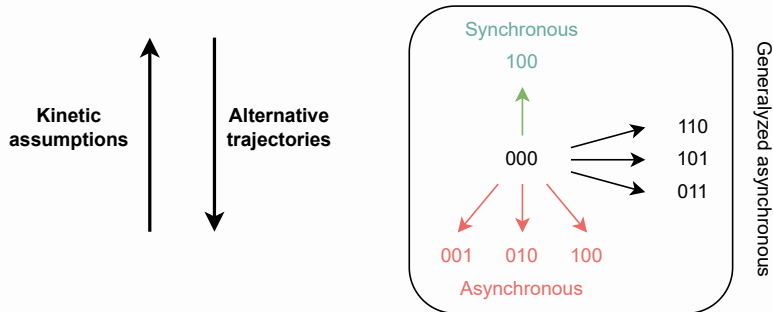


Synchronous updating (deterministic)



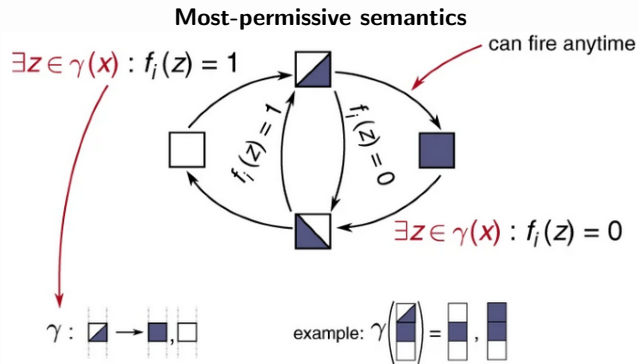
No quantitative parameters but requires a precise description of competing effects
(rates, updating scheme, etc.)

Explosion of number of transitions



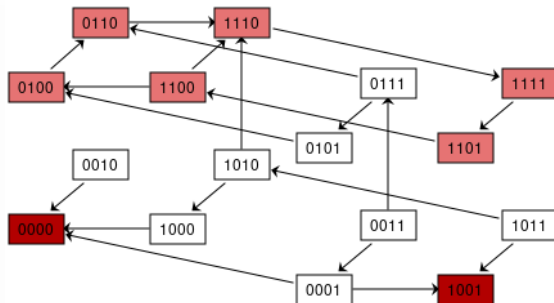
In absence of kinetic knowledge, many alternative trajectories are possible

Explosion of number of transitions



(Pauleve et al., *Nat. Comm.* 2020)

Useful to compute over-approximations of reachability properties



Methods

- Stable states: fixed points of the dynamics
Constraint solving $\exists x : f(x) = x$
- Complex attractors: cycles of the dynamics
Symbolic exploration $\exists C : \forall x \in C : F(x) = C$
- Trap spaces: stable patterns of the dynamics
Constraint solving $\exists C : \forall x \in C : f(x) \in C$

(Naldi et al., *CMSB* 2007)

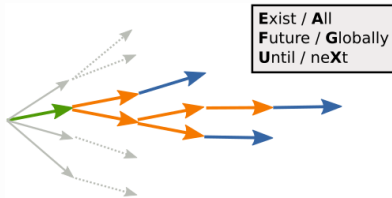
(Garg et al., *Bioinformatics* 2008)

(Klarner et al., *Nat Comput* 2015)

Identification of reachabilities ($A \rightarrow B$)

- Explicitly generating the whole STG (CPU/memory inefficient)
- Symbolic model-checking (NuSMV, ...)

(Monteiro et al., *WODES* 2014)



- Static analysis on prime-implicant graph (Pint) efficient/approximation

(Pauleve et al., *Math Structures Comput Sci* 2012)

$x = 00000$



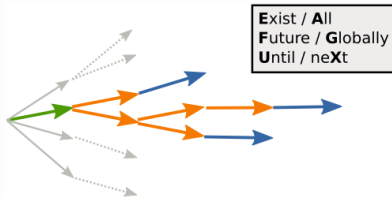
Yes!
? Maybe?
No!

$p = 10*1*$

Identification of reachabilities ($A \rightarrow B$)

- Explicitly generating the whole STG (CPU/memory inefficient)
- Symbolic model-checking (NuSMV, ...)

(Monteiro et al., *WODES* 2014)



See Ben Hall's 2024 lecture!
in git history :)

- Static analysis on prime-implicant graph (Pint) efficient/approximation

(Pauleve et al., *Math Structures Comput Sci* 2012)

$x = 00000$

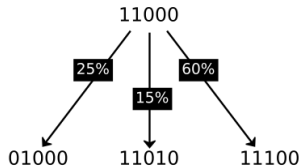


Yes!
? Maybe?
No!

$p = 10*1*$

Quantification of reachability

Probabilities of transition
(firing rates)



Stochastic simulations

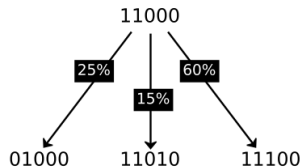
Estimated distribution
of state's probabilities



- **Avatar** (Mendes et al., *Frontiers in Physiology* 2018) @ (bioLQM&GINsim)
- **MaBoSS** (Stoll et al., *BMC Syst Biol* 2012)

Quantification of reachability

Probabilities of transition
(firing rates)



Stochastic simulations

Estimated distribution
of state's probabilities

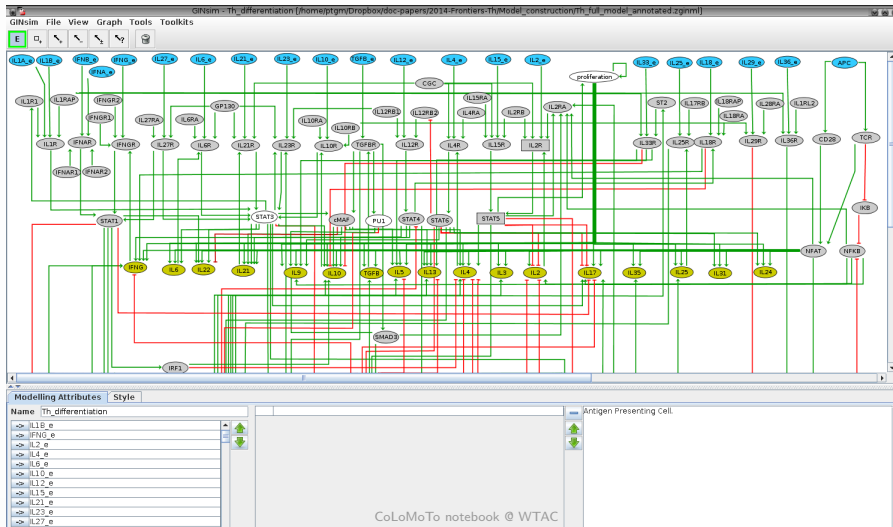


- **Avatar** (Mendes et al., *Frontiers in Physiology* 2018) @(bioLQM&GINsim)
- **MaBoSS** (Stoll et al., *BMC Syst Biol* 2012)

See Arnau's lecture!

Gene Interaction Network **simulation** is a computer tool for the modeling and simulation of genetic regulatory networks implementing the **logical formalism**

<http://ginsim.github.io>



Gene Interaction Network simulation is a computer tool for the modeling and simulation of genetic regulatory networks implementing the **logical formalism**

<http://ginsim.github.io>

Analysis methods

Static Analysis:

- Stable state search
(Naldi *et al.*, *CMSB* 2007)
- Circuit analysis
(Thieffry, *Brief. Bioinform.* 2007)
- Model reduction
(Naldi *et al.*, *Theor. Comput. Sci.* 2011)

Dynamics:

- STG construction
- HTG construction
(Bérengruer *et al.*, *Chaos* 2013)
- Priority classes
(Fauré *et al.*, *Bioinform.* 2006)
- Perturbations
- Reachability (**explicit**)

Gene Interaction Network simulation is a computer tool for the modeling and simulation of genetic regulatory networks implementing the **logical formalism**

<http://ginsim.github.io>

Analysis methods

Static Analysis:

- Stable state search
(Naldi *et al.*, *CMSB* 2007)
- Circuit analysis
(Thieffry, *Brief. Bioinform.* 2007)
- Model reduction
(Naldi *et al.*, *Theor. Comput. Sci.* 2011)

Dynamics:

- STG construction
- HTG construction
(Bérengruer *et al.*, *Chaos* 2013)
- Priority classes
(Fauré *et al.*, *Bioinform.* 2006)
- Perturbations
- Reachability (**explicit**)

Many other tools exist, though!

Trajectory analysis
NuSMV

Network editor
Cell Designer

Model building
CaSQ

Knowledge base
Simulations
The Cell Collective

Attractors
Reachability
Pint

Modification
Visualization
Attractors
bioLQM
GINsim

BoolNet
Simulation
Generation
Delays

MaBoSS
Stochastic
simulations
Attractors

PyBoolNet
Attractors
Simulations

BoolSim
Attractors
Symbolic
reachability

CellNopt
Model fitting
(data analysis)

How to use all these tools together? different formats, installations, versions, etc.

Problems: Interoperability and Reproducibility

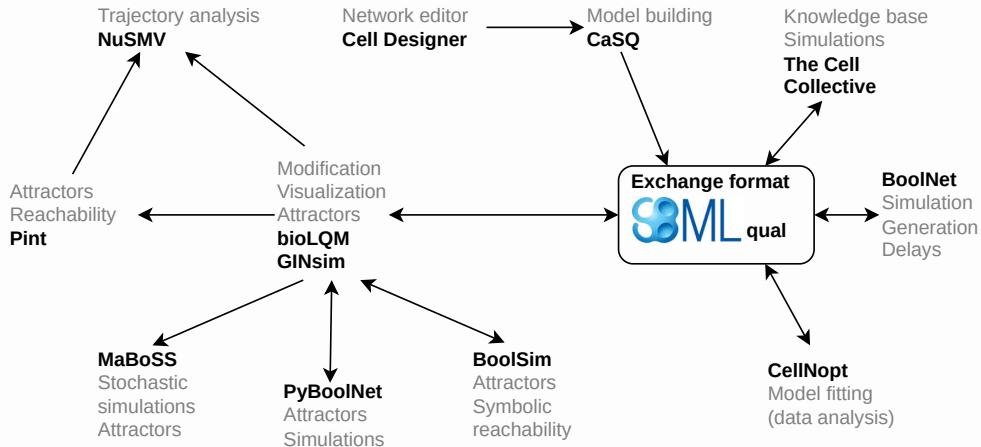
[Home](#)[Members](#)[Events](#)

SBML qual

SBML qual is an extension of the **Systems Biology Markup Language (SBML)** Level 3 standard, designed for the representation of multivalued qualitative models of biological networks. The first SBML qual proposal in 2008 led to the creation of the CoLoMoTo consortium. The extension was accepted by the SBML community in 2011, after consulting the wider community and refining the original proposal through additional meetings with developers of various related software tools.

The final specification was approved by the SBML Editors in the spring of 2013.





Disclaimer: not a complete list of tools!

Consortium for Logical **M**odels and **T**ools



Install

Conda packages, Docker image



Perform analysis

Python API for all tools
Jupyter: semi-interactive interface



Share

Analysis: Jupyter notebooks
Models: SBML qual exchange format



Try it online (No installation)
Limited resources, download to save

tmpnb.colomoto.org


colomoto-docker (requires docker and Python)
Python wrapper hiding complex docker commands

Pure Docker

Raw Docker commands

Conda packages

Linux only, less reproducible

(Naldi et al., *Frontiers in Physiology* 2018)

Maintained by:

Initially



Aurélien Naldi

Sporadically



Pedro Monteiro

Mainly

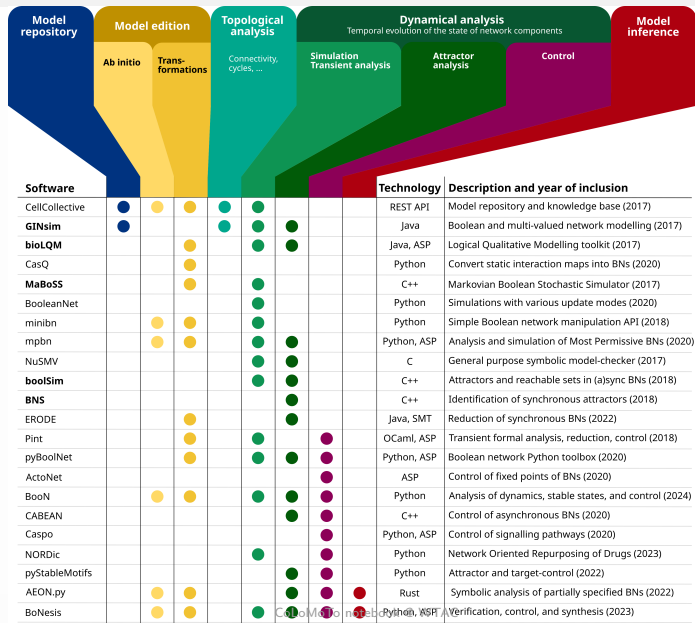


Loic Paulevé

BoNesis	https://github.com/bnediction/bonesis	Synthesis of Boolean Networks from architecture and dynamical properties	Python module bonesis
BooleanNet	https://github.com/ialbert/booleannet	Simulation of Boolean regulatory networks	Python module boolean2
boolSim	https://www.vital-it.ch/research/software/boolSim	Attractors and reachable sets in synchronous and asynchronous Boolean networks	Python module boolsim
CABEAN	https://satoss.uni.lu/software/CABEAN/	A Software Tool for the Control of Asynchronous Boolean Networks	Python module cabean
Caspo	https://bioasp.github.io/caspo/	Reasoning on the response of logical signaling networks with Answer Set Programming	Python module caspo_control
CaSQ	https://github.com/soli/casq	Convert static interaction maps into executable models	Python module casq
CellCollective	https://cellcollective.org	Model repository and knowledge base	Python module cellcollective
ERODE	https://github.com/colomoto/ERODE-CoLoMoTo	Backward Boolean Equivalence of Boolean networks	Python module erode
GINsim	http://ginsim.org	Boolean and multi-valued network modelling	Python module ginsim

Initial release: 6 software tools (Naldi et al., *Frontiers in Physiology* 2018)

Current release: 25 software tools! (Noël et al., *Interface Focus* 2025)



Problems

- Installation problems (OS, dependencies, ...)
- Interaction between tools (formats, versions, ...)
- Correct order of steps/commands
- Specific tool parameters
- ...

CoLoMoTo docker/notebook

- **Integrated** access to (pre-installed) tools
- **Interoperability** between tools
- **Reproducibility** of analyses

<https://github.com/colomoto/colomoto-docker>

Support: Linux, MacOS, Windows

Requirements: Docker, Python

Installation (or update)

```
pip install -U colomoto-docker
```

Conda

CoLoMoTo is also available using conda, but package dependencies cannot be fully controlled.

Reproducibility is better ensured using docker.

Also, some tools executables are not available for Windows.

Usage (start the notebook)

<code>colomoto-docker</code>	uses the most recently local image
<code>colomoto-docker -V latest</code>	fetches the latest published image
<code>colomoto-docker -V 2025-05-01</code>	fetches a specific image (one / month)

<code>colomoto-docker --bind .</code>	the notebook uses the current directory
---------------------------------------	---

```
colomoto-docker -V latest --bind ~/some/work/directory/
```

CoLoMoTo Notebook will open in your browser

Load model

```
[1]: 1 import ginsim
      2 # Model URL: http://ginsim.org/node/4
      3 lrg = ginsim.load("http://ginsim.org/sites/default/files/boolean_cell_cycle.zginml")
```

This notebook has been executed using the docker image `colomoto/colomoto-docker:2024-04-01`

Downloading http://ginsim.org/sites/default/files/boolean_cell_cycle.zginml

Do not forget attaching "boolean_cell_cycle.zginml" file with your notebook

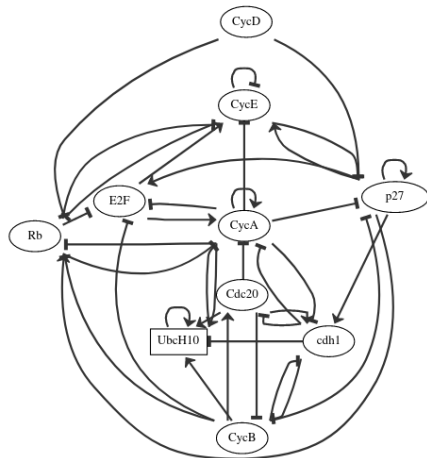
Using local file [boolean_cell_cycle.zginml](#)

(model is downloaded and saved in the work directory)

Show LRG

```
[2]: 1 ginsim.show(lrg)
```

```
[2]:
```



Attractor identification

bioLQM

```
[9]: 1 from colomoto_jupyter import tabulate
      2 import biolqm
      3 lqm = ginsim.to_biolqm(lrg)
      4 lqm_fixpoints = biolqm.fixpoints(lqm)
      5 tabulate(lqm_fixpoints)
```

```
[9]:
```

	CycD	Rb	E2F	CycE	CycA	p27	Cdc20	cdh1	UbcH10	CycB
	0	0	1	0	0	1	0	1	0	0

Attractor identification

bioLQM

```
[9]: 1 from colomoto_jupyter import tabulate
      2 import biolqm
      3 lqm = ginsim.to_biolqm(lrg)
      4 lqm_fixpoints = biolqm.fixpoints(lqm)
      5 tabulate(lqm_fixpoints)
```

```
[9]:
```

	CycD	Rb	E2F	CycE	CycA	p27	Cdc20	cdh1	UbcH10	CycB
	0	0	1	0	0	1	0	1	0	0

Pint

```
[11]: 1 from colomoto_jupyter import tabulate
      2 import pypint
      3 pm = ginsim.to_pint(lrg)
      4 pm_fixpoints = pm.fixpoints()
      5 tabulate(pm_fixpoints)
```

```
[11]:
```

	CycD	Rb	E2F	CycE	CycA	p27	Cdc20	cdh1	UbcH10	CycB
	0	0	1	0	0	1	0	1	0	0

Attractor identification (trapspace)

bioLQM

```
[12]: 1 trapspace = biolqm.trapspace(lqm)
      2 tabulate(trapspace)
```

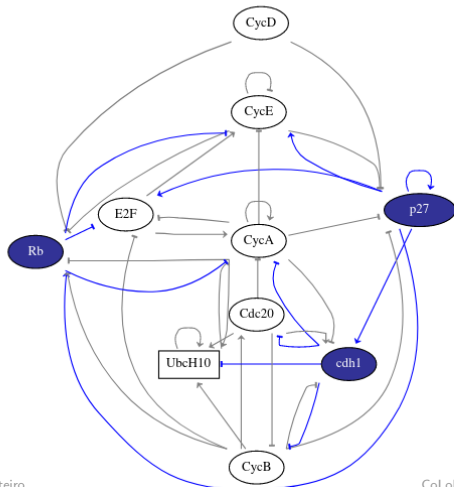
```
[12]:
```

	CycD	Rb	E2F	CycE	CycA	p27	Cdc20	cdh1	UbcH10	CycB
0	0	1	0	0	0	1	0	1	0	0
1	1	0	254	254	254	0	254	254	254	254

Map attractors on model LRG (stable states)

```
[14]: 1 ginsim.show(lrg, lqm_fixpoints[0])
```

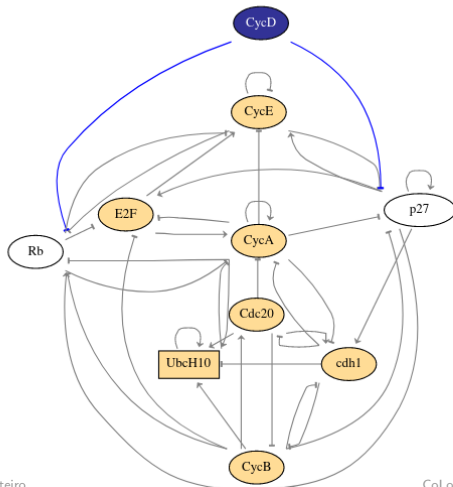
```
[14]:
```



Map attractors on model LRG (trap spaces)

```
[15]: 1 ginsim.show(lrg, trapspaces[1])
```

```
[15]:
```



Model perturbations

bioLQM

```
[18]: 1 lqm_p27_KO = biolqm.perturbation(lqm, "p27%0")
      2 # lqm_p27_range = biolqm.perturbation(lqm, "p27%1:2")
      3 lqm_CycD_p27 = biolqm.perturbation(lqm, "CycD:p27%0")
```

Pint

```
[20]: 1 pm_p27_KO = pm.lock(p27=0)
```

MaBoSS

```
[21]: 1 mb_p27_KO = mb.mutate("p27", "OFF")
```

Model reduction

bioLQM

```
[24]: 1 lqm_reduced = biolqm.reduce(lqm, "fixed, outputs")
      2 tabulate(biolqm.fixpoints(lqm_reduced))
```

(Naldi et al., *TCS* 2011)

(Naldi et al., *CMSB* 2012)

Formal reachability analysis

Pint

```
1 an = biolqm.to_pint(lqm)
2 target_state = {"Apoptosis":1}
3 an.reachability(goal=target_state)
```

True

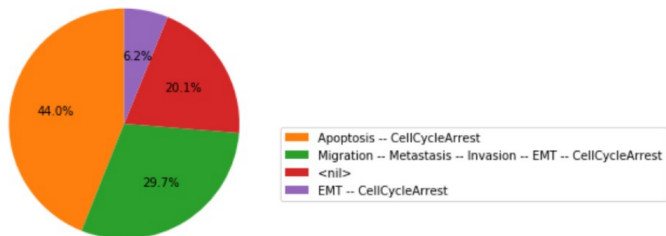
NuSMV model checker

```
1 smv = an.to_nusmv()
2 ctl_specs = {
3     "stable-apoptosis": EF(AG(S(Apoptosis=1)))
4 }
5 smv.add_ctls(ctl_specs)
6 smv.verify()
```

{'stable-apoptosis': False}

Estimation of reachability probabilities

```
1 masim = biolqm.to_maboss(lqm)
2 mares = masim.run()
3 mares.plot_piechart()
```



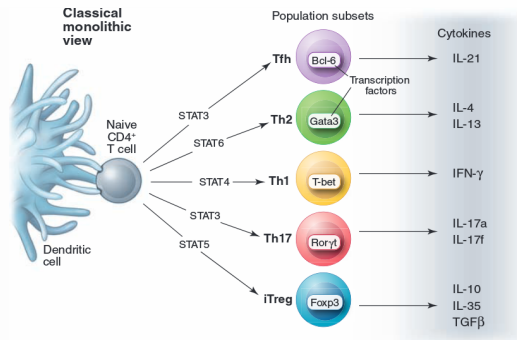


Mechanisms Underlying Lineage Commitment and Plasticity of Helper CD4⁺ T Cells

John J. O'Shea* and William E. Paul

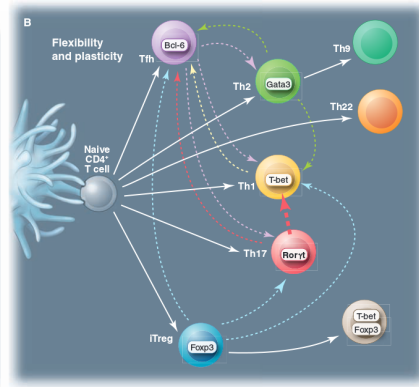
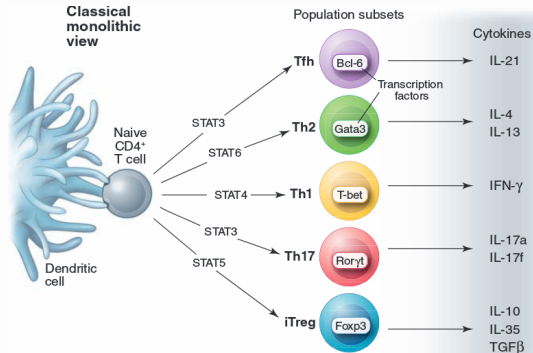
CD4⁺ T cells are critical for host defense but are also major drivers of immune-mediated disease. These T cells specialize to become distinct subsets and produce restricted patterns of cytokines, which are tailored to combat various microbial pathogens. Although classically viewed as distinct lineages, recent work calls into question whether helper CD4⁺ T cell subsets are more appropriately viewed as terminally differentiated cells or works in progress. Herein, we review recent advances that pertain to this topic and the mechanisms that contribute to helper CD4⁺ T cell commitment and plasticity. The therapeutic implications of these new findings are also considered.

(O'Shea and Paul, *Science* 2010)



- T-helper (CD4⁺) lymphocytes play a role in the regulation of immune responses
- Faced with pathogens, naive CD4⁺ T cells differentiate into distinct Th cell populations expressing distinct cytokine profiles
- Polarization into cell types is governed by transcription factors

(O'Shea and Paul, *Science* 2010)

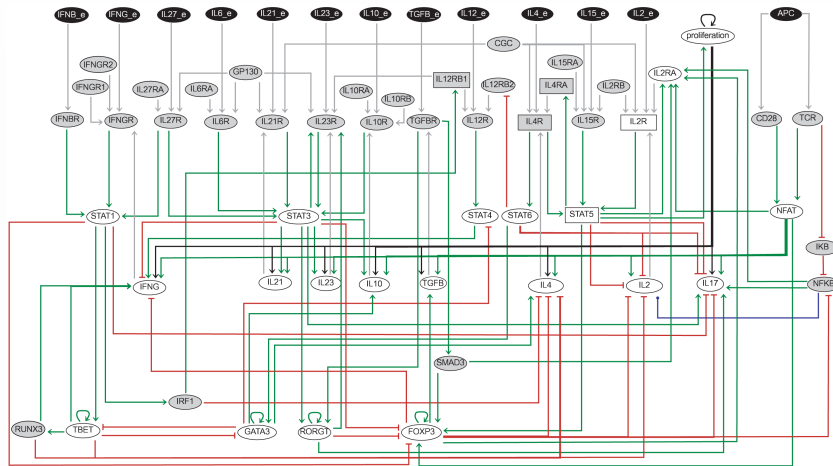


Objective

- Model capable of reproducing Th cell differentiation

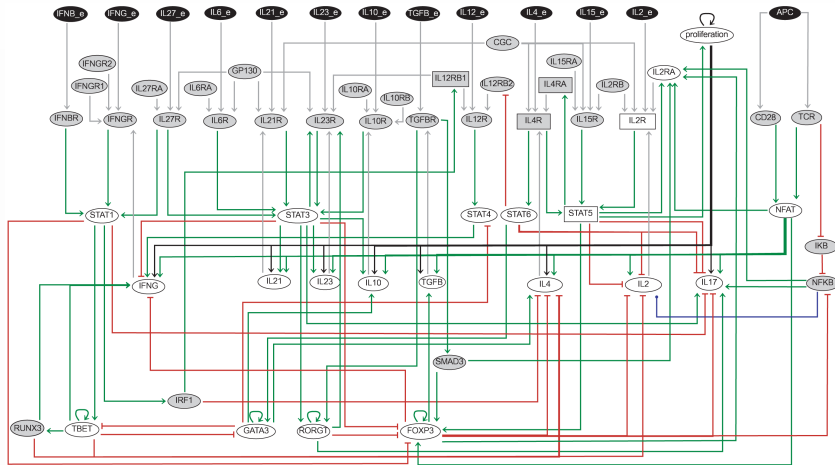
(Naldi et al., *PLoS Comput. Biol.* 2010)

- Capable of exhibiting plasticity behaviours



(Naldi et al., *PLoS Comp Biol* 2010)

Model size: 64 components (13 inputs + 51 internal) and 138 interactions



(Naldi et al., *PLoS Comp Biol* 2010)

Model size: 64 components (13 inputs + 51 internal) and 138 interactions

STG size: 2^{64} states!!

⇒ **Too large to perform meaningful simulations!**

Approach to analyse the T-helper cell differentiation model:

- Identify the biologically relevant **states** corresponding to **Th cell subtypes**
- Compute the **reprogramming and plasticity** between **Th cell subtypes**

Start by computing the set of biologically relevant **Th cell subtypes**

Cell lineage	Master regulators			
	TBET	GATA3	RORGT	FOXP3
Th0	0	0	0	0
Th1	1	0	0	0
Th17	0	0	1	0
Th2	0	1	0	0
Treg	0	0	0	1
Th1 Foxp3+	1	0	0	1
Th2 Foxp3+	0	1	0	1
Treg ROR γ t+	0	0	1	1
Th1 Foxp3+ ROR γ t+	1	0	1	1
Th2 Foxp3+ ROR γ t+	0	1	1	1
Th1 ROR γ t+	1	0	1	0
Th2 ROR γ t+	0	1	1	0

(Naldi et al., *PLoS Comp Biol* 2010)

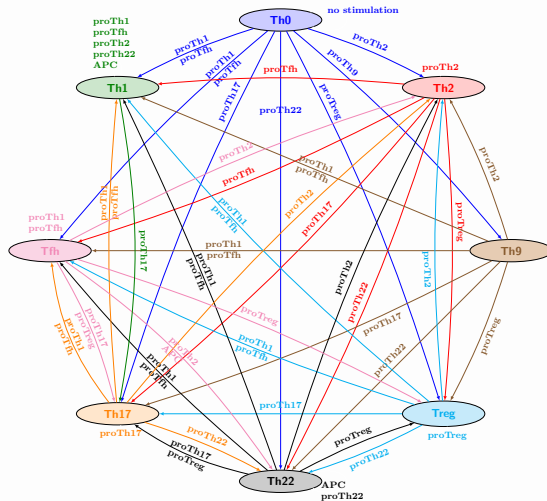
Then characterize the set of prototypic **environmental conditions**

Tile code	description	Inputs						
		APC	IL2_e	IL4_e	IL6_e	IL10_e	IL12_e	TGFB_e
□	no stimulation							
■	APC only							
■	pro-Th1 (i)							
■	pro-Th1 (ii)							
■	pro-Th2							
■	pro-Th17							
■	pro-Treg (i)							
■	pro-Treg (ii)							

(Naldi et al., *PLoS Comp Biol* 2010)

Case study - Reachability analysis of relevant Th cell subtypes

Use **model checking** to systematically explore the reachability of all **Th cell subtypes** under all **environmental/input conditions**. Obtaining a “reprogramming map” between Th cell subtypes.



Model repository

<http://ginsim.github.io/models/>



Gene Interaction Network simulation



Search

Gene Interaction Network
simulation

Home

Model Repository

Documentation

Companion Tools

Installation

Contact

Model Repository

Model name	Taxon - all terms	Process - all terms
Lysis vs lysogeny decision in the phage lambda	Phage Lambda	Lysis-lysogeny decision
Gap Model	D. melanogaster	Segmentation, Development
The pair-rule cross-regulatory module	D. melanogaster	Segmentation, Development
Controlling tryptophan biosynthesis	E. Coli, Bacterium	Tryptophan biosynthesis
DV boundary formation of the Wing imaginal disc	D. melanogaster	Wing Imaginal disk,

- Load the Th model from GINsim model repository

<https://ginsim.github.io/sites/default/files/Frontiers-Th-Full-model-annotated.zginml>

https://ginsim.github.io/sites/default/files/Th_differentiation_reduced_model.zginml

(Naldi *et al.*, *PLos Comp Biol* 2010)

but for practical reasons (time/memory) we'll use the older/simpler model

<https://ginsim.github.io/models/2006-mammal-th-differentiation/>

https://ginsim.github.io/sites/default/files/Th_17.zginml

(Mendoza, *Biosystems* 2006)

- Display the regulatory graph
- Compute all stable states using the `biolqm` (and then `Pint`) python modules
- Visualise attractor on LRG
- Perform reachability analysis using `boolsim`, `NuSMV`, `Pint`, ...

Volume 15, Issue 3

22 August 2025

[< Previous Article](#)[Next Article >](#)**Article contents**

Abstract

1 Introduction

Reproducible Boolean model analyses and simulations with the CoLoMoTo software suite: a tutorial

Vincent Noël ; Aurélien Naldi ; Laurence Calzone  ; Loïc Paulevé  ; Denis Thieffry  + **Author & article information***Interface Focus* (2025) 15 (3): 20250002 .<https://doi.org/10.1098/rsfs.2025.0002> **Article history** 

Abstract

This tutorial provides stepwise instructions to install over 20 tools, written in multiple languages. Their integration in the *CoLoMoTo* software suite makes them accessible with a single popular language (*Python*), thereby enabling reproducible and sophisticated dynamical analyses of logical models of complex cellular networks. The tutorial specifically focuses on the analysis of a previously published model of the regulatory network controlling mammalian cell proliferation. It includes chunks of *Python* code to reproduce several of the results and figures published in the original article, and further extends these results with the help of selected tools included in the *CoLoMoTo* suite. The tutorial covers the visualization of the network with the tool *GINsim*, an attractor analysis with *bioLQM*, the computation of synchronous attractors with *BNS*, the extraction of modules from the full model, stochastic simulations of the wild-type model and of selected perturbations with *MaBoSS* and finally the delineation of compressed probabilistic state transition graphs. The integration of all these analyses in an executable *Jupyter Notebook* greatly eases their reproducibility, as well as the inclusion of further extensions. The notebook provided along with this tutorial further constitutes a template, which can be enriched with other *CoLoMoTo* tools, to develop comprehensive dynamical analyses of various biological network models.

https://nbviewer.org/urls/zenodo.org/records/15602428/files/CoLoMoTo_Tutorial_Sizek_2025.ipynb
(also included in the git repository)



Curie, Paris



CoLoMoTo



INRIA, Paris



IBENS, Paris



I2M, Marseille



LaBRI, Bordeaux



IGC, Oeiras

Contacts

<http://pedromonteiro.org>

Pedro.Tiago.Monteiro@tecnico.ulisboa.pt